

CS 551 Systems Programming

Lecture 8: Traits

You have already been using these

Last lecture you wrote `#[derive(Debug)]` and got `{:?}` formatting for free. You did not ask what that meant.

It meant: **implement the `Debug` trait for this type.**

A trait is Rust's answer to "what do several different types have in common?" It is the only mechanism for shared behavior in the language, and once you see it, `derive`, `{}`, `==`, `?`, and `for` loops all turn out to be the same idea wearing different hats.

Today's path

1. **What a trait is.** Shared behavior, declared and implemented.
2. **derive**, **explained.** What those attributes were writing for you.
3. **Conversion traits.** **From** and **Into**, which you will use constantly.
4. **The rules.** The orphan rule, and the newtype escape hatch.

Next lecture is generics, which is what lets you write *one* function that works on every type implementing a trait. Today is the trait itself.

1. What a trait is

A trait is a set of method signatures

```
1 trait Describe {  
2     fn describe(&self) -> String;  
3 }
```

That declares a capability: "a type that is `Describe` can produce a `String` describing itself."

It defines no data and creates no values. It is a **contract**. Types opt in by implementing it.

If you have written a Java interface or a C++ abstract base class, this is the familiar part. The differences show up shortly.

Implementing it

```
1 trait Describe {
2     fn describe(&self) -> String;
3 }
4
5 struct Rectangle { width: u32, height: u32 }
6 struct Circle { radius: f64 }
7
8 impl Describe for Rectangle {
9     fn describe(&self) -> String {
10         format!("{}", self, self.height)
11     }
12 }
13
14 impl Describe for Circle {
15     fn describe(&self) -> String {
16         format!("circle of radius {}", self.radius)
17     }
18 }
19
20 fn main() {
21     println!("{}", Rectangle { width: 3, height: 4 }.describe());
22     println!("{}", Circle { radius: 1.5 }.describe());
23 }
```

impl Trait for Type

Read the syntax literally: *implement this trait, for this type*.

Note what is **not** happening:

- `Rectangle` does not mention `Describe` in its definition. The struct was written first; the trait can be bolted on later, by different code.
- There is no base class, no vtable pointer in the struct, and no runtime cost. `Rectangle` is still two `u32`s.
- A type can implement any number of traits.

This is the big structural difference from inheritance. **The type does not have to know about the trait in advance.**

Traits can supply default bodies

```
1 trait Describe {
2     fn name(&self) -> String;
3
4     fn describe(&self) -> String {
5         format!("a thing called {}", self.name())    // default
6     }
7 }
8
9 struct Rectangle;
10
11 impl Describe for Rectangle {
12     fn name(&self) -> String {
13         "rectangle".to_string()
14     }
15     // describe() comes free
16 }
17
18 fn main() {
19     println!("{}", Rectangle.describe());
20 }
```

A large surface from a small core

A default body can call **other methods of the same trait**, including ones with no default at all.

That is the whole trick: an implementor writes the two or three methods that genuinely differ per type, and inherits the twenty built on top of them.

`Iterator` is the extreme case, with one required method and dozens of provided ones. [<https://doc.rust-lang.org/std/iter/trait.Iterator.html>]

And an impl can override a default

```
1 trait Describe {
2     fn name(&self) -> String;
3     fn describe(&self) -> String { format!("a thing called {}", self.name()) }
4 }
5
6 struct Rectangle;
7 impl Describe for Rectangle {
8     fn name(&self) -> String { "rectangle".to_string() }
9     fn describe(&self) -> String { "a very fine rectangle".to_string() }
10 }
11
12 fn main() {
13     println!("{}", Rectangle.describe());    // the override wins
14 }
```

Required methods, default methods, overriding a default. If that sounds like inheritance, it is, in the behavior sense. What Rust lacks is inheritance of **data**.

Supertraits: a trait that requires another

```
1 trait Named {
2     fn name(&self) -> String;
3 }
4
5 trait Greet: Named {                // requires Named
6     fn greet(&self) -> String {
7         format!("hello, {}", self.name()) // so it may call Named's methods
8     }
9 }
10
11 struct Rex;
12 impl Named for Rex {
13     fn name(&self) -> String { "Rex".to_string() }
14 }
15 impl Greet for Rex {}
16
17 fn main() {
18     println!("{}", Rex.greet());
19 }
```

The `:` reads as "requires"

`trait Greet: Named` means any type implementing `Greet` **must** also implement `Named`.

That is the closest thing Rust has to `extends`, and the difference matters: it is a requirement on the implementor, not inherited behavior. `Greet` gets to *call* `Named`'s methods; it gets none of its data.

2. `derive`, explained

What `#[derive(Debug)]` writes

`derive` is a macro that generates an `impl` block for you. These two are equivalent in effect:

```
1 #[derive(Debug)]
2 struct Point { x: i32, y: i32 }
```

```
1 struct Point { x: i32, y: i32 }
2
3 impl std::fmt::Debug for Point {
4     fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
5         // ... generated, field by field ...
6     }
7 }
```

That is the whole trick.

Why `derive` sometimes refuses

`Debug`, `Clone`, `PartialEq`, `Hash` and the rest are **ordinary traits**.

`derive` writes the obvious implementation when one exists.

It is only *possible* when every field also implements the trait, because the generated body works field by field. That is why a struct containing a `String` cannot derive `Copy`: `String` is not `Copy`, so there is no obvious implementation to write.

Display is the one you write by hand

```
1 use std::fmt;
2
3 struct Point { x: i32, y: i32 }
4
5 impl fmt::Display for Point {
6     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
7         write!(f, "({}, {})", self.x, self.y)
8     }
9 }
10
11 fn main() {
12     let p = Point { x: 3, y: 4 };
13     println!("{p}");
14     println!("{}", p.to_string());
15 }
```

Display is the one you write by hand

```
1 use std::fmt;
2
3 struct Point { x: i32, y: i32 }
4
5 impl fmt::Display for Point {
6     fn fmt(&self, f: &mut fmt:
7         write!(f, "({}, {})", self.x, self.y)
8     }
9 }
10
11 fn main() {
12     let p = Point { x: 3, y: 4 };
13     println!("{p}");
14     println!("{}", p.to_string());
15 }
```

impl Trait for Type. This is the whole shape of implementing a trait

And **to_string()** now exists too.
You never wrote it

There is no `#[derive(Display)]`, on purpose. Rust makes you say it.

Debug vs Display

	<code>{:?}</code> Debug	<code>{}</code> Display
Audience	you, debugging	your user
Derivable	yes	no, by design
Should exist on	almost everything	types with one obvious rendering
Projects	required for error types	required for error messages

`to_string()` **comes free**. Implement `Display` and you get `.to_string()` automatically, because the standard library implements `ToString` for every type implementing `Display`. That is a *blanket implementation*, which is next lecture's material.

Both matter in the projects: `thiserror` generates a `Display` impl from your `#[error("...")]` attributes, and requires `Debug`.

3. The conversion traits

From, and the one you get for free

From says how to build one thing from another

```
1 struct Celsius(f64);
2 struct Fahrenheit(f64);
3
4 impl From<Celsius> for Fahrenheit {
5     fn from(c: Celsius) -> Self {
6         Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
7     }
8 }
9
10 fn main() {
11     let f = Fahrenheit::from(Celsius(100.0));
12     println!("{}", f.0);           // 212
13 }
```

Read `impl From<Celsius> for Fahrenheit` as: "you can build a `Fahrenheit` **from** a `Celsius`."

You have used this since Lecture 1. `String::from("hi")` is exactly this trait.

Implement `From`, get `Into` free

```
1 fn main() {
2     let a: Fahrenheit = Celsius(100.0).into();    // Into, never implemented
3     let b = Fahrenheit::from(Celsius(0.0));      // From, what we wrote
4
5     println!("{}", a.0, b.0);
6 }
```

We never wrote `impl Into<Fahrenheit> for Celsius`. The standard library provides it for free, for every `From` impl that exists.

Implement `From`. Never implement `Into`. The free direction only flows one way.

Note that `.into()` needs to know the target type, which is why `a` carries an annotation. That is the same inference you saw with `parse` in Lecture 1.

Why you should care: this is what ? runs on

```
1 fn read_config() -> Result<Config, MyError> {  
2     let text = std::fs::read_to_string("config.toml");  
3 }
```

This foreshadows the error-handling lecture, and it is the single most useful thing From does.

Why you should care: this is what `?` runs on

```
1 fn read_config() -> Result<Config, MyError> {  
2     let text = std::fs::read_to_string("config.toml");  
3 }
```

`read_to_string` returns an `io::Error`, but the function returns `MyError`. `?` calls `From` to convert it

This foreshadows the error-handling lecture, and it is the single most useful thing `From` does.

Error handling and `impl From<T>`

The previous slide compiles **only if** `impl From<io::Error> for MyError` exists. The compiler says so outright:

```
the question mark operation (`?`) implicitly performs a conversion
on the error value using the `From` trait
```

`?` is not magic. It is an early return plus a `From::from` call on the error. Every custom error type you write in this course exists to have `From` impls hung off it.

TryFrom, when the conversion can fail

```
1 struct Even(i32);
2 impl TryFrom<i32> for Even {
3     type Error = String; // associated type; a type the implementor chooses
4
5     fn try_from(n: i32) -> Result<Self, Self::Error> {
6         match n % 2 { 0 => Ok(Even(n)), _ => Err(format!("{n} is odd")),
7         }
8     }
9 }
10 fn main() {
11     println!("{:?}", Even::try_from(4).map(|e| e.0));
12     println!("{:?}", Even::try_from(7).map(|e| e.0));
13 }
```

From is for conversions that always succeed. TryFrom returns a Result and gives you try_into() for free, the same way From gives you into().

4. The rules

The orphan rule

You may implement a trait for a type only if **you own the trait, or you own the type**.

```
1 use std::fmt;
2
3 impl fmt::Display for Vec<i32> {
4     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
5         write!(f, "a vector")
6     }
7 }
```

```
error[E0117]: only traits defined in the current crate can be
             implemented for types defined outside of the crate
```

`Display` belongs to the standard library. `Vec` belongs to the standard library. You own neither, so the impl is forbidden.

Why the rule exists

Without it, two different crates could each implement `Display for Vec<i32>`. Pulling in both would be ambiguous, with no way to resolve it. The rule guarantees at most one impl of a trait for a type exists anywhere in a program.

That property is called **coherence**, and a lot of Rust rests on it.

You will hit this rule for real in the projects, the first time you want to implement something on a `Vec` or a `String` you did not define. The fix is on the next slide, and you already know it.

The newtype escape hatch

Wrap the foreign type in a struct you own, and now you own the type:

```
1 use std::fmt;
2
3 struct Ids(Vec<i32>);           // a type I own, wrapping one I do not
4
5 impl fmt::Display for Ids {
6     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
7         write!(f, "{} ids", self.0.len())
8     }
9 }
10
11 fn main() {
12     println!("{}", Ids(vec![1, 2, 3]));    // 3 ids
13 }
```

This is the newtype pattern from Lecture 7, doing its second job. It costs nothing at runtime: `Ids` has the same layout as the `Vec` it wraps.

The tradeoff is that you no longer get the `Vec` methods automatically. You expose the ones you want, or implement `Deref`.

LIVE CODING CHALLENGE

Take `struct Meters(f64)`. Get `Debug`, `Clone`, and `PartialEq` without writing a single impl body. Then hand-write `Display` so it prints `9.8 m`, and `From<f64>` so that `9.8.into()` gives you a `Meters`.

Pause here. Try it before the next slide.

The whole picture on one type

```
1 use std::fmt;
2
3 #[derive(Debug, Clone, PartialEq)]
4 struct Meters(f64);
5
6 impl fmt::Display for Meters {
7     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
8         write!(f, "{} m", self.0)
9     }
10 }
11
12 impl From<f64> for Meters {
13     fn from(v: f64) -> Self { Meters(v) }
14 }
15
16 fn main() {
17     let a: Meters = 9.8.into();
18     let b = a.clone();
19
20     println!("{a}");           // Display: 9.8 m
21     println!("{a:?}");        // Debug: Meters(9.8)
22     println!("{}", a == b);   // PartialEq: true
23 }
```

Recap

1. **A trait is a contract:** method signatures a type can promise to provide.
2. **`impl Trait for Type`**. The type does not have to know about the trait in advance, which is the structural difference from inheritance.
3. **Default bodies** let a trait offer a lot from a small required core, and an impl may override them.
4. **Supertraits** (`trait A: B`) require another trait. This is inheritance of behavior; Rust only lacks inheritance of data.
5. **`derive` writes an `impl` for you**, when there is an obvious one. `Display` has none, so you write it.
6. **Implement `From`, get `Into` free**. And `?` is `From` on the error type.
7. **The orphan rule:** own the trait or own the type. **Newtype** is the way around it.

Exercises

1. Add a `Describe` impl for a third shape. Then give the trait a second method with a default body that uses the first, and override it for exactly one type.
2. Implement `Display` for the `Rectangle` from Lecture 7 so it prints `3x4`. Then call `.to_string()` on it without implementing `ToString`. Why does that work?
3. Write `impl From<Rectangle> for (u32, u32)`. Now try `impl From<(u32, u32)> for Rectangle`. One of these may be forbidden. Predict which, then check, then explain.
4. Try to `impl Default for String`. Read the error and name the rule. Then make it work with a newtype.
5. Write `TryFrom<&str> for Meters` that fails on unparseable input. What did you pick for `type Error`, and why?
6. Add `#[derive(PartialOrd)]` to `Meters` and sort a `Vec<Meters>`. Now remove `PartialEq` and read the error. What does that tell you about how the derives relate?

Reading

- **The Rust Programming Language**, chapter 10.2 (Traits) and chapter 20.2 for the advanced material.
- **Rust by Example**, the traits section, and the `From`/`Into` section.
- **The `std::convert` docs**. Short, and `From`/`Into`/`TryFrom` are four of the ten most useful traits in the language.

Next lecture: generics. One function, every type that implements the trait, and no runtime cost.

Next: Lecture 9

Generics and Dispatch

Writing it once, and what the compiler does about it.